

```
# %%capture
!pip install langchain==0.2.5 faiss-cpu==1.8.0 cohere==5.5.8 langchain-community=
!pip install llama-cpp-python==0.2.78 --extra-index-url https://abetlen.github.io

## IMPORTANT: Make sure to restart the session after installing the packages above
```

```
Building wheel for llama-cpp-python (pyproject.toml) ... done
Created wheel for llama-cpp-python: filename=llama_cpp_python-0.2.78-cp312-cp312-
Stored in directory: /root/.cache/pip/wheels/ee/f8/a8/978da1e57d80f11bb70643e0e0c
Successfully built llama-cpp-python
ERROR: Operation cancelled by user
```

```
import cohere

# Paste your API key here. Remember to not share publicly
api_key = '#####'

# Create and retrieve a Cohere API key from os.cohere.ai
co = cohere.Client(api_key)
```

```
text = """
Interstellar is a 2014 epic science fiction film co-written, directed, and produced
It stars Matthew McConaughey, Anne Hathaway, Jessica Chastain, Bill Irwin, Ellen Bur
Set in a dystopian future where humanity is struggling to survive, the film follows

Brothers Christopher and Jonathan Nolan wrote the screenplay, which had its origins
Caltech theoretical physicist and 2017 Nobel laureate in Physics[4] Kip Thorne was
Cinematographer Hoyte van Hoytema shot it on 35 mm movie film in the Panavision anar
Principal photography began in late 2013 and took place in Alberta, Iceland, and Los
Interstellar uses extensive practical and miniature effects and the company Double I

Interstellar premiered on October 26, 2014, in Los Angeles.
In the United States, it was first released on film stock, expanding to venues using
The film had a worldwide gross over $677 million (and $773 million with subsequent
It received acclaim for its performances, direction, screenplay, musical score, visu
It has also received praise from many astronomers for its scientific accuracy and po
Interstellar was nominated for five awards at the 87th Academy Awards, winning Best

# Split into a list of sentences
texts = text.split('.')

# Clean up to remove empty spaces and new lines
texts = [t.strip(' \n') for t in texts]
```

✓ 2. Embedding the Text Chunks

```
import numpy as np

# Get the embeddings
response = co.embed(
    texts=texts,
    input_type="search_document",
).embeddings

embeds = np.array(response)
print(embeds.shape)
```

(15, 4096)

3. Building The Search Index

```
import faiss

dim = embeds.shape[1]
index = faiss.IndexFlatL2(dim)
index.add(np.float32(embeds))
```

4. Search the index

```
import pandas as pd

def search(query, number_of_results=3):

    # 1. Get the query's embedding
    query_embed = co.embed(texts=[query],
                             input_type="search_query",).embeddings[0]

    # 2. Retrieve the nearest neighbors
    distances, similar_item_ids = index.search(np.float32([query_embed]), number_of_results)

    # 3. Format the results
    texts_np = np.array(texts) # Convert texts list to numpy for easier indexing
    results = pd.DataFrame(data={'texts': texts_np[similar_item_ids[0]],
                                'distance': distances[0]})

    # 4. Print and return the results
    print(f"Query: '{query}'\nNearest neighbors:")
    return results
```

```
query = "how precise was the science"
results = search(query)
results
```

```
Query: 'how precise was the science'
Nearest neighbors:
```

	texts	distance
0	It has also received praise from many astronom...	10757.379883
1	Caltech theoretical physicist and 2017 Nobel l...	11566.131836
2	Interstellar uses extensive practical and mini...	11922.833008

```
from rank_bm25 import BM25Okapi
from sklearn.feature_extraction import _stop_words
import string
```

```
def bm25_tokenizer(text):
    tokenized_doc = []
    for token in text.lower().split():
        token = token.strip(string.punctuation)

        if len(token) > 0 and token not in _stop_words.ENGLISH_STOP_WORDS:
            tokenized_doc.append(token)
    return tokenized_doc
```

```
from tqdm import tqdm

tokenized_corpus = []
for passage in tqdm(texts):
    tokenized_corpus.append(bm25_tokenizer(passage))

bm25 = BM25Okapi(tokenized_corpus)
```

100%|██████████| 15/15 [00:00<00:00, 38908.20it/s]

```
def keyword_search(query, top_k=3, num_candidates=15):
    print("Input question:", query)

    ##### BM25 search (lexical search) #####
    bm25_scores = bm25.get_scores(bm25_tokenizer(query))
    top_n = np.argpartition(bm25_scores, -num_candidates)[-num_candidates:]
    bm25_hits = [{ 'corpus_id': idx, 'score': bm25_scores[idx]} for idx in top_n]
    bm25_hits = sorted(bm25_hits, key=lambda x: x['score'], reverse=True)

    print(f"Top-3 lexical search (BM25) hits")
    for hit in bm25_hits[0:top_k]:
        print("Wt{:.3f}Wt{}".format(hit['score'], texts[hit['corpus_id']].replace("\n", " ")))
```

```
keyword_search(query = "how precise was the science")
```

```
Input question: how precise was the science
Top-3 lexical search (BM25) hits
1.789 Interstellar is a 2014 epic science fiction film co-written, directed
1.373 Caltech theoretical physicist and 2017 Nobel laureate in Physics[4] by
0.000 It stars Matthew McConaughey, Anne Hathaway, Jessica Chastain, Bill
```

✓ Caveats of Dense Retrieval

```
query = "What is the mass of the moon?"
results = search(query)
results
```

Query: 'What is the mass of the moon?'

Nearest neighbors:

	texts	distance
0	Cinematographer Hoyte van Hoytema shot it on 3...	12854.458984
1	The film had a worldwide gross over \$677 milli...	13301.030273
2	It has also received praise from many astron...	13332.011719

✓ Reranking Example

```
query = "how precise was the science"
results = co.rerank(query=query, documents=texts, top_n=3, return_documents=True)
results.results
```

```
[RerankResponseResultsItem(document=RerankResponseResultsItemDocument(text='It has also received praise from many astronomers for its scientific accuracy and portrayal of theoretical astrophysics'), index=12, relevance_score=0.1698185),
 RerankResponseResultsItem(document=RerankResponseResultsItemDocument(text='The film had a worldwide gross over $677 million (and $773 million with subsequent re-releases), making it the tenth-highest grossing film of 2014'), index=10, relevance_score=0.07004896),
 RerankResponseResultsItem(document=RerankResponseResultsItemDocument(text='Caltech theoretical physicist and 2017 Nobel laureate in Physics[4] Kip Thorne was an executive producer, acted as a scientific consultant, and wrote a tie-in book, The Science of Interstellar'), index=4, relevance_score=0.0043994132)]
```

```
for idx, result in enumerate(results.results):
    print(idx, result.relevance_score , result.document.text)
```

```
0 0.1698185 It has also received praise from many astronomers for its scientific accu
1 0.07004896 The film had a worldwide gross over $677 million (and $773 million with
2 0.0043994132 Caltech theoretical physicist and 2017 Nobel laureate in Physics[4] K
```

```
def keyword_and_reranking_search(query, top_k=3, num_candidates=10):
    print("Input question:", query)

    ##### BM25 search (lexical search) #####
    bm25_scores = bm25.get_scores(bm25_tokenizer(query))
    top_n = np.argpartition(bm25_scores, -num_candidates)[-num_candidates:]
    bm25_hits = [{'corpus_id': idx, 'score': bm25_scores[idx]} for idx in top_n]
    bm25_hits = sorted(bm25_hits, key=lambda x: x['score'], reverse=True)

    print(f"Top-3 lexical search (BM25) hits")
    for hit in bm25_hits[0:top_k]:
        print("Wt{:.3f}Wt{}".format(hit['score'], texts[hit['corpus_id']]).replace("\n", " "))

    #Add re-ranking
    docs = [texts[hit['corpus_id']] for hit in bm25_hits]
```

```
print(f"\nTop-3 hits by rank-API ({len(bm25_hits)} BM25 hits re-ranked)")
results = co.rerank(query=query, documents=docs, top_n=top_k, return_documents=True)
for hit in results.results:
    print("Wt{:.3f}Wt{}".format(hit.relevance_score, hit.document.text.replace(
```

```
keyword_and_reranking_search(query = "how precise was the science")
```

Input question: how precise was the science

Top-3 lexical search (BM25) hits

```
1.789  Interstellar is a 2014 epic science fiction film co-written, directed
1.373  Caltech theoretical physicist and 2017 Nobel laureate in Physics[4] h
0.000  Interstellar uses extensive practical and miniature effects and the d
```

Top-3 hits by rank-API (10 BM25 hits re-ranked)

```
0.004  Caltech theoretical physicist and 2017 Nobel laureate in Physics[4] h
0.004  Set in a dystopian future where humanity is struggling to survive, th
0.003  Brothers Christopher and Jonathan Nolan wrote the screenplay, which h
```

✓ Retrieval-Augmented Generation

✓ Example: Grounded Generation with an LLM API

```
query = "income generated"

# 1- Retrieval
# We'll use embedding search. But ideally we'd do hybrid
results = search(query)

# 2- Grounded Generation
docs_dict = [{'text': text} for text in results['texts']]
response = co.chat(
    message = query,
    documents=docs_dict
)

print(response.text)
```

Query: 'income generated'

Nearest neighbors:

The film generated a worldwide gross of over \$677 million, or \$773 million with subse

response

```
NonStreamedChatResponse(text='The film generated a worldwide gross of over $677
million, or $773 million with subsequent re-releases.', generation_id='bebc32bd-
d620-42cf-bd13-f8d1f96d4aa6', citations=[ChatCitation(start=21, end=57,
text='worldwide gross of over $677 million', document_ids=['doc_0']),
ChatCitation(start=62, end=103, text='$773 million with subsequent re-releases.',
document_ids=['doc_0'])], documents=[{'id': 'doc_0', 'text': 'The film had a
worldwide gross over $677 million (and $773 million with subsequent re-releases),
```

```
making it the tenth-highest grossing film of 2014'}}], is_search_required=None,
search_queries=None, search_results=None, finish_reason='COMPLETE', tool_calls=None,
chat_history=[Message_User(message='income generated', tool_calls=None,
role='USER'), Message_Chatbot(message='The film generated a worldwide gross of over
$677 million, or $773 million with subsequent re-releases.', tool_calls=None,
role='CHATBOT')], prompt=None,
meta=ApiMeta(api_version=ApiMetaApiVersion(version='1', is_deprecated=None,
is_experimental=None), billed_units=ApiMetaBilledUnits(input_tokens=106,
output_tokens=26, search_units=None, classifications=None),
tokens=ApiMetaTokens(input_tokens=797, output_tokens=95), warnings=None),
response_id='319002db-505d-4d3c-903f-41eebf0f856')
```

```
response.citations
```

```
[ChatCitation(start=21, end=57, text='worldwide gross of over $677 million',
document_ids=['doc_0']),
ChatCitation(start=62, end=103, text='$773 million with subsequent re-releases.',
document_ids=['doc_0'])]
```

✓ Example: RAG with Local Models

✓ Loading the Generation Model

```
!wget https://huggingface.co/microsoft/Phi-3-mini-4k-instruct-gguf/resolve/main/Phi-
--2025-12-05 13:06:07-- https://huggingface.co/microsoft/Phi-3-mini-4k-instruct-gguf/
Resolving huggingface.co (huggingface.co)... 13.35.202.97, 13.35.202.40, 13.35.202.12
Connecting to huggingface.co (huggingface.co)|13.35.202.97|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://cas-bridge.xethub.hf.co/xet-bridge-us/662698108f7573e6a6478546/df22
--2025-12-05 13:06:07-- https://cas-bridge.xethub.hf.co/xet-bridge-us/662698108f7573
Resolving cas-bridge.xethub.hf.co (cas-bridge.xethub.hf.co)... 13.33.45.119, 13.33.45.
Connecting to cas-bridge.xethub.hf.co (cas-bridge.xethub.hf.co)|13.33.45.119|:443...
HTTP request sent, awaiting response... 200 OK
Length: 2393231072 (2.2G)
Saving to: 'Phi-3-mini-4k-instruct-q4.gguf'

Phi-3-mini-4k-instr 100%[=====>] 2.23G 23.3MB/s in 98s

2025-12-05 13:07:46 (23.2 MB/s) - 'Phi-3-mini-4k-instruct-q4.gguf' saved [2393231072]
```

```
from langchain import LlamaCpp

# Make sure the model path is correct for your system!
llm = LlamaCpp(
    model_path="Phi-3-mini-4k-instruct-q4.gguf",
    n_gpu_layers=-1,
    max_tokens=500,
    n_ctx=2048,
    seed=42,
```

```
verbose=False
)
```

✓ Loading the Embedding Model

```
from langchain.embeddings.huggingface import HuggingFaceEmbeddings

# Embedding Model for converting text to numerical representations
embedding_model = HuggingFaceEmbeddings(
    model_name='BAAI/bge-small-en-v1.5'
)
```

```
/tmp/ipython-input-1728599694.py:4: LangChainDeprecationWarning: The class `HuggingFaceEmbeddings` is deprecated. Use `HuggingFaceHubEmbeddings` instead.
embedding_model = HuggingFaceEmbeddings(
/usr/local/lib/python3.12/dist-packages/sentence_transformers/cross_encoder/CrossEncoder.py:10: DeprecationWarning: Please use `sentence_transformers.CrossEncoder` instead.
from tqdm.autonotebook import tqdm, trange
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning: The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models.
warnings.warn(
modules.json: 100% 349/349 [00:00<00:00, 28.9kB/s]
config_sentence_transformers.json: 100% 124/124 [00:00<00:00, 14.6kB/s]
README.md: 94.8k/? [00:00<00:00, 8.24MB/s]
sentence_bert_config.json: 100% 52.0/52.0 [00:00<00:00, 4.72kB/s]
config.json: 100% 743/743 [00:00<00:00, 67.0kB/s]
model.safetensors: 100% 133M/133M [00:03<00:00, 82.6MB/s]
tokenizer_config.json: 100% 366/366 [00:00<00:00, 42.4kB/s]
vocab.txt: 232k/? [00:00<00:00, 15.7MB/s]
tokenizer.json: 711k/? [00:00<00:00, 32.4MB/s]
special_tokens_map.json: 100% 125/125 [00:00<00:00, 14.2kB/s]
config.json: 100% 190/190 [00:00<00:00, 21.8kB/s]
```

✓ Preparing the Vector Database

```
from langchain.vectorstores import FAISS

# Create a local vector database
db = FAISS.from_texts(texts, embedding_model)
```


The RAG Prompt

```
from langchain import PromptTemplate
from langchain.chains import RetrievalQA

# Create a prompt template
template = """<|user|>
Relevant information:
{context}

Provide a concise answer the following question using the relevant information prov
{question}<|end|>
<|assistant|>"""
prompt = PromptTemplate(
    template=template,
    input_variables=["context", "question"]
)

# RAG Pipeline
rag = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type='stuff',
    retriever=db.as_retriever(),
    chain_type_kwargs={
        "prompt": prompt
    },
    verbose=True
)
```

```
rag.invoke('Income generated')
```

```
> Entering new RetrievalQA chain...
```

```
> Finished chain.
```

```
{'query': 'Income generated',
 'result': " Interstellar grossed over $677 million worldwide in 2014 and had
additional earnings from subsequent re-releases, totaling approximately $773
million. The film's release utilized both traditional film stock and digital
projectors across various venues to maximize its income generation potential."}
```

